

Counting Sort

Table of contents

Counting Sort	1
Counting Sort	1
Solution	3
Duplicati	5
Solution	6
Array di strutture	8
Descrizione dei Passi	8
Solution	14
Complessità computazionale (tempo)	16
FONTI PRINCIPALI	17

Counting Sort

Counting Sort

Algoritmo di ordinamento di **complessità lineare** utilizzabile solo sotto determinate ipotesi

Esempio banale:

Ordinare n interi presi nell'intervallo $[1, n]$ senza duplicati.

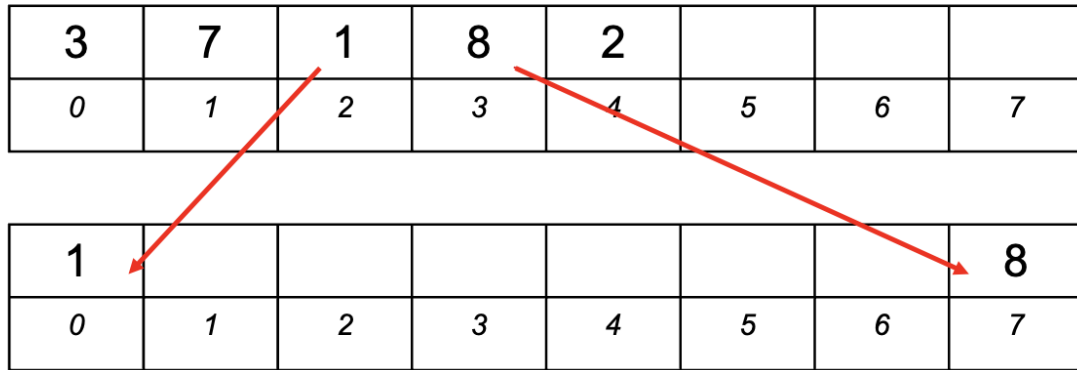
La soluzione è banale: è il vettore $[1, 2, \dots, n]$.

-
- Caso k elementi, $k < n$,

Cosa succede se abbiamo k elementi, con $k < n$, sempre senza duplicati?

Counting Sort è un algoritmo di ordinamento che, dato un insieme di **numeri interi distinti entro un intervallo noto**, ordina gli elementi **determinando direttamente la posizione finale** di ciascun valore in base alla sua grandezza relativa.

Idea base: identifico i valori massimi e minimi (**max** e **min**). Associo **min** all'indice 0 e **max** all'indice **max-min**.



Definito in questo modo il *mapping* tra numeri e indici, posso usarlo per spostare gli altri valori.

Esempio

dato il vettore

`vett[] = {3, 7, 1, 8, 2}`

1) Costruisco il vettore **presence** delle presenze

Riscalco ciascun valore di **vett** rispetto al minimo. Il valore riscalato è l'indice di **presence**, in cui inserisco 1 se **value** è presente nel vettore originale.

```
// Mark the presence of each element
for (i = 0; i < nEl; i++) {
    value = vett[i];
    idx = value - min;
    presence[idx] = 1;
}
```

Otengo i seguenti valori

presence	1	1	1	0	0	0	1	1
idx	0	1	2	3	4	5	6	7
value	1	2	3	4	5	6	7	8

Si noti come il vettore originario contenga 5 valori nel range $[1, 8]$. Il valore dell'indice j (che indicizza **presence**) va da 0 a 7.

2) scorro il vettore **presence** ed inserisco il valore corretto nell'array finale.

```
// Build the sorted array
idx = 0;
for (i = 0; i < range; i++) {
    if (presence[i] == 1) {
        vett[idx] = i + min;
        idx++;
    }
}
```

Ottengo

vett[] = {1, 2, 3, 7, 8}

Solution

```
// Sorting/counting_sort_1

#include <stdio.h>
#include <stdlib.h>

void printArray(int *v, int n) {
    printf("[");
    for (int i = 0; i < n; i++) {
        printf("%d ", v[i]);
    }
    printf("]\n");
}
```

```

void counting_sort(int *vett, int nEl) {
    int i, j, idx;
    int min = vett[0], max = vett[0], value;
    for (i = 1; i < nEl; i++) {
        // determino il range
        min = (vett[i] < min) ? vett[i] : min;
        max = (vett[i] > max) ? vett[i] : max;
    };

    int range = max - min + 1;
    int presence[range];
    //int *presence = malloc(range * sizeof(*presence ));
    for (i = 0; i < range; i++) presence[i] = 0;

    // start snippet presence
    // Mark the presence of each element
    for (i = 0; i < nEl; i++) {
        value = vett[i];
        idx = value - min;
        presence[idx] = 1;
    }
    // end snippet presence

    // start snippet build_array
    // Build the sorted array
    idx = 0;
    for (i = 0; i < range; i++) {
        if (presence[i] == 1) {
            vett[idx] = i + min;
            idx++;
        }
    }
    // end snippet build_array

    // Free the allocated memory
    // free(presence);

}

int main() {

```

```

int vett[] = {3, 7, 1, 8, 2};
int nEl = sizeof(vett) / sizeof(vett[0]);
printArray(vett, nEl);
counting_sort(vett, nEl);

printArray(vett, nEl);

return 0;
}

```

Duplicati

Cosa succede se abbiamo duplicati? Occorre modificare il mapping. L'elemento **min** corrisponderà sempre all'indice 0. Gli altri elementi vanno riscalati per 'far posto' ai duplicati.

Consideriamo un vettore di lunghezza n , con soli k valori distinti

Esempio:

`vett[] = {3, 7, 1, 8, 2, 7, 3, 8}`

$n = 8$; $k = 5$ valori distinti

1) Costruisco il vettore **count** delle occorrenze

`vett[] = {3, 7, 1, 8, 2, 7, 3, 8}`

Riscalco ciascun valore di **vett** rispetto al minimo. Il valore riscalato (**idx**) è l'indice di **count**.

```

// Count the occurrences of each element
for (i = 0; i < nEl; i++) {
    value = vett[i];
    idx = value - min; // remap
    count[idx]++;
}

```

Ottengo i seguenti valori

count	1	1	2	0	0	0	2	2
idx	0	1	2	3	4	5	6	7
value	1	2	3	4	5	6	7	8

- 2) scorro il vettore `count` ed inserisco il valore corretto nell'array finale, tante volte quante sono indicate da `count`

```
// Build the sorted array
i = 0;
for (idx = 0; idx < range; idx++) {
    while (count[idx] > 0) {
        vett[i] = idx + min;
        i++;
        count[idx]--;
    }
}
```

Ottengo

`vett[] = {1, 2, 3, 3, 7, 7, 8, 8}`

Solution

```
// Sorting/counting_sort_2

#include <stdio.h>
#include <stdlib.h>

// DUPLICATES
void printArray(int *v, int n) {
    printf("[");
    for (int i = 0; i < n; i++) {
        printf("%d ", v[i]);
    }
    printf("]\n");
}

void counting_sort(int *vett, int nEl) {
    int i, j, idx, value;
    int min = vett[0], max = vett[0];
```

```

    for (i = 1; i < nEl; i++) {
        if (vett[i] < min) min = vett[i];
        if (vett[i] > max) max = vett[i];
    }

    int range = max - min + 1;
    int *count = malloc(range * sizeof(*count));
    for (i = 0; i < range; i++) count[i] = 0;

    // start snippet count
    // Count the occurrences of each element
    for (i = 0; i < nEl; i++) {
        value = vett[i];
        idx = value - min; // remap
        count[idx]++;
    }
    // end snippet count

    // start snippet build_array
    // Build the sorted array
    i = 0;
    for (idx = 0; idx < range; idx++) {
        while (count[idx] > 0) {
            vett[i] = idx + min;
            i++;
            count[idx]--;
        }
    }
    // end snippet build_array

    // Free the allocated memory
    free(count);
}

int main() {
    int vett[] = {3, 7, 1, 8, 2, 7, 3, 8};
    int nEl = sizeof(vett) / sizeof(vett[0]);
    printArray(vett, nEl);
    counting_sort(vett, nEl);
    printArray(vett, nEl);
    return 0;
}

```

Array di strutture

Se devo ordinare strutture (e non solo numeri) l'algoritmo va leggermente modificato.

```
// Sort Structures
typedef struct {
    int index; // The integer field used for sorting
    int v1;
    float v2;
    // other fields
} Element;
```

Supponiamo che l'ordinamento si basi sul campo **index**. Due strutture con lo stesso valore di **index** sono comunque **strutture diverse** e come tali vanno trattate. Non mi basta, come nel caso precedente, indicare solo **quante** sono le strutture con un certo valore di **index**. Devo anche distinguerle tra loro.

Consideriamo i seguenti dati in **vett**. Ciascun elemento è una struttura.

.index	3	7	1	8	2	7	3	8
.v1	100	200	300	400	500	600	700	800
.v2	1.1	2.2	3.3	4.4	5.5	6.6	7.7	8.8

Descrizione dei Passi

1. **Inizializzazione delle strutture di supporto** Si costruisce un array **count** per memorizzare il numero di **occorrenze cumulate** di ciascun valore della chiave **index**

```
// Count the occurrences of each element
for (i = 0; i < nEl; i++) {
    value = vett[i].index;
    idx = value - min; // remap
    count[idx]++;
}

// Cumulative occurrences
for (idx = 1; idx < range; idx++) {
    count[idx] += count[idx - 1];
}
```


Si alloca inoltre un array **output** della stessa dimensione del vettore originale, in cui verranno inseriti i record ordinati. Le **occorrenze cumulate** servono a determinare in modo diretto la **posizione finale** di ciascun elemento nel vettore **output**.

[illegible]

Perché mi occorrono le occorrenze cumulate?

Per ciascun indice devo sapere quante posizioni devo occupare - quanti elementi devo posizionare.

Esempio. Consideriamo l'indice 7, che è l'ultimo indice **utile** dell'array . L'array deve contenere 8 elementi, quindi l'ultimo elemento della sequenza deve essere posizionato in $8-1=7$. Se all'indice 7 avessimo trovato 100, allora l'array avrebbe dovuto contenere 100 elementi, e l'ultimo elemento avrebbe dovuto essere posizionato in $100-1=99$ ***

2. Scansione inversa dell'array di input e determinazione della posizione corretta

Si scorre l'array di input **vett a ritroso** (cioè da destra verso sinistra). Questo passaggio è cruciale per garantire la **stabilità** dell'algoritmo: tra elementi con chiavi uguali, viene mantenuto l'ordine relativo originale.

```
// Build the sorted array
Element *output = malloc(nEl * sizeof(*output));
for (i = nEl - 1; i >= 0; i--) {
    idx = vett[i].index - min;
    k = count[idx] - 1;
    output[k] = vett[i];
    count[idx]--;
}
```

Per ogni elemento in esame, si prende il valore della chiave (`.index`) riscaldato e lo si usa per accedere all'array `count`.

.index	3	7	1	8	2	7	3	8
.v1	100	200	300	400	500	600	700	800
.v2	1.1	2.2	3.3	4.4	5.5	6.6	7.7	8.8
						(2)	(1)	

era 4

count (cum)	1	2	3	4	4	4	6	7
(idx)	0	1	2	3	4	5	6	7

output				3				8
				700				800
				7.7				8.8
				(2)				(1)

min=1

indice in count index-min	indice in output count-1
2	3

.index	3	7	1	8	2	7	3	8	min=1	
.v1	100	200	300	400	500	600	700	800		
.v2	1.1	2.2	3.3	4.4	5.5	6.6	7.7	8.8		
					(3)	(2)	(1)			
									indice in count	indice in output
									index-min	count-1
									6	5
count (cum)	1	2	3	4	4	4	5	7		
(idx)	0	1	2	3	4	5	6	7		
output				3		7		8		
				700		600		800		
				7.7		6.6		8.8		
				(2)		(3)		(1)		

[illegible][illegible]

.index	3	7	1	8	2	7	3	8		min=1	
.v1	100	200	300	400	500	600	700	800			
.v2	1.1	2.2	3.3	4.4	5.5	6.6	7.7	8.8			
			(6)	(5)	(4)	(3)	(2)	(1)			
count (cum)	0	1	3	4	4	4	5	6		indice in count index-min	indice in output count-1
(idx)	0	1	2	3	4	5	6	7		0	0
output	1	2		3		7	8	8			
	300	500		700		600	400	800			
	3.3	5.5		7.7		6.6	4.4	8.8			
	(6)	(4)		(2)		(3)	(5)	(1)			

.index	3	7	1	8	2	7	3	8		min=1	
.v1	100	200	300	400	500	600	700	800			
.v2	1.1	2.2	3.3	4.4	5.5	6.6	7.7	8.8			
	(7)	(6)	(5)	(4)	(3)	(2)	(1)				
count (cum)	0	1	3	4	4	4	4	6		indice in count index-min	indice in output count-1
(idx)	0	1	2	3	4	5	6	7		6	4
output	1	2		3	7	7	8	8			
	300	500		700	200	600	400	800			
	3.3	5.5		7.7	2.2	6.6	4.4	8.8			
	(6)	(4)		(2)	(7)	(3)	(5)	(1)			

.index	3	7	1	8	2	7	3	8		min=1	
.v1	100	200	300	400	500	600	700	800			
.v2	1.1	2.2	3.3	4.4	5.5	6.6	7.7	8.8			
	(8)	(7)	(6)	(5)	(4)	(3)	(2)	(1)			
										indice in count	indice in output
										index-min	count-1
										2	2
count (cum)	0	1	2	4	4	4	4	6			
(idx)	0	1	2	3	4	5	6	7			
output	1	2	3	3	7	7	8	8			
	300	500	100	700	200	600	400	800			
	3.3	5.5	1.1	7.7	2.2	6.6	4.4	8.8			
	(6)	(4)	(8)	(2)	(7)	(3)	(5)	(1)			

Solution

```
// Sorting/counting_sort_3

#include <stdio.h>
#include <stdlib.h>

// start snippet structures
// Sort Structures
typedef struct {
    int index; // The integer field used for sorting
    int v1;
    float v2;
    // other fields
} Element;
// end snippet structures

void printArray(Element *v, int n) {
    printf("[");
    for (int i = 0; i < n; i++) {
        printf("(%d %d %0.2f )", v[i].index, v[i].v1, v[i].v2 );
    }
    printf("\n");
}
```

```

}

void counting_sort(Element *vett, int nEl) {
    int i, j, k, value, idx;
    int min = vett[0].index, max = vett[0].index;
    for (i = 1; i < nEl; i++) {
        if (vett[i].index < min) min = vett[i].index;
        if (vett[i].index > max) max = vett[i].index;
    }

    int range = max - min + 1;
    int *count = malloc(range * sizeof(*count));
    for (int i = 0; i < range; i++) count[i] = 0;

    // start snippet count
    // Count the occurrences of each element
    for (i = 0; i < nEl; i++) {
        value = vett[i].index;
        idx = value - min; // remap
        count[idx]++;
    }

    // Cumulative occurrences
    for (idx = 1; idx < range; idx++) {
        count[idx] += count[idx - 1];
    }
    // end snippet count

    // start snippet build_array
    // Build the sorted array
    Element *output = malloc(nEl * sizeof(*output));
    for (i = nEl - 1; i >= 0; i--) {
        idx = vett[i].index - min;
        k = count[idx] - 1;
        output[k] = vett[i];
        count[idx]--;
    }
    // end snippet build_array
    // Copy the sorted elements back to the original array - NEW
    for (i = 0; i < nEl; i++) {
        vett[i] = output[i];
    }
}

```

```

    // Free the allocated memory
    free(count);
    free(output);
}

int main() {
    Element vett[] = {
        {3, 100, 1.1},
        {7, 200, 2.2},
        {1, 300, 3.3},
        {8, 400, 4.4},
        {2, 500, 5.5},
        {7, 600, 6.6},
        {3, 700, 7.7},
        {8, 800, 8.8}
    };
    int nEl = sizeof(vett) / sizeof(vett[0]);

    printf("Original array:\n");
    printArray(vett, nEl);

    counting_sort(vett, nEl);

    printf("\nSorted array:\n");
    printArray(vett, nEl);

    return 0;
}

```

Complessità computazionale (tempo)

Consideriamo sempre un vettore di lunghezza n , con chiavi distribuite in un range k

Il Counting Sort si articola in tre fasi principali

- Calcolo delle occorrenze: costo $O(n)$
- Calcolo delle occorrenze cumulate: costo $O(k)$
- Posizionamento degli elementi: costo $O(n)$

dunque la complessità computazionale è $O(n + k)$

L'algoritmo è stabile.

Un **algoritmo di ordinamento stabile** è un algoritmo di ordinamento che preserva l'ordine relativo degli elementi con chiavi uguali durante il processo di ordinamento. In altre parole, se due elementi hanno lo stesso valore della chiave di ordinamento, la loro posizione relativa nell'array ordinato rimane la stessa di quella nell'array originale.

- Algoritmi Stabili: Merge Sort, Insertion Sort, Bubble Sort.
- Algoritmi Non stabili: QuickSort, HeapSort, Selection Sort

La stabilità è una proprietà rilevante negli ordinamenti multi-criterio, in cui una sequenza viene ordinata successivamente secondo più chiavi. In particolare, se un algoritmo di ordinamento è stabile, l'applicazione di un ordinamento su una seconda chiave v_2 preserva l'ordine relativo determinato da una precedente ordinazione sulla chiave v_1 , tra gli elementi che condividono lo stesso valore di v_2 . Questo comportamento è fondamentale per garantire la coerenza del risultato finale nei processi di sorting gerarchico.

Si noti che, se si scorre l'array di input `vett` da **sinistra verso destra**, l'algoritmo **perderebbe la stabilità**.

Questo accade perché l'array `count` contiene **posizioni cumulative finali**: per ogni valore della chiave, indica la **posizione dell'ultimo elemento** con quella chiave nell'array `output`.

Scorrendo `vett` da **destra verso sinistra**, ogni volta che si incontra un elemento con chiave k , lo si colloca nella **posizione indicata da `count[k] - 1`**, e poi si decrementa `count[k]`. In questo modo, l'**ultimo elemento** con chiave k viene posizionato per primo (in fondo), seguito dal penultimo, e così via: **l'ordine originale tra elementi uguali viene preservato**, garantendo la stabilità.

Al contrario, scorrendo da **sinistra verso destra**, i **primi elementi** con chiave k occuperebbero le **ultime posizioni disponibili**, che dovrebbero invece essere riservate agli ultimi elementi. Ciò porta all'**inversione dell'ordine originario tra elementi uguali**, rendendo l'ordinamento **non stabile**.

(provate)

FONTI PRINCIPALI

Sedgewick